Git Basics

Version control for your project, from Zed

Why version control

When you write code over several days, you keep changing files. Without a record, you cannot tell what changed, when, or why, and you cannot go back to a version that worked. **Version control** is a tool that saves snapshots of your project so you always have a history you can return to. In this course we use git, the standard tool for this, and GitHub, a website that stores a copy of your project online so a partner can see it too.

You do not need to understand how git works inside. You need five simple actions, and you can do all of them from Zed, the editor you have used since Session VII. Zed has a git panel (a side panel that shows your changes) and, like everything in Zed, git actions are available from the **command palette** (`Cmd+Shift+P` on macOS, `Ctrl+Shift+P` on Windows/Linux). This page uses the command-palette names: they always work, whatever the panel looks like. We show the matching terminal command next to each action as well, because the words are the same everywhere, and you will see those commands in videos, tutorials, and error messages. Learn the word once and it works in both places.

> **Note**
> **In Session IX** we do the one-time setup below together in class: install git and `gh`, log in once. Everything else on this page is what we do together in Session X.

The five operations

Below are the only five git operations you need for this course. Each one has a plain definition, the way to do it in Zed, the matching terminal command, and when you actually need it.

Repository

A **repository** (or "repo") is a folder that git watches. Everything inside it, and its whole history, is tracked together.

In this course a repository **starts on GitHub**: you create it on the website (with a README), then **clone** it onto your computer (see Clone). Inside that cloned folder, `uv init --no-package` adds the Python files. Because the folder came from GitHub, it is already a repo and already knows where to push.

The only time you make a repo by hand is an existing folder that is *not* yet under git. In Zed you run **`git: init`** from the command palette. The terminal equivalent is:

```
git init
```

When you need it: almost never. Use `git init` only to add git to an old folder that does not have it yet.

Commit

A **commit** is one saved snapshot of your project, with a short message describing what you changed. Think of it as a labeled save point you can always come back to.

A commit has two small steps: first you **stage** the changes you want to include (mark them as "part of this snapshot"), then you commit them with a message.

In Zed you run **`git: stage all`** from the command palette, type a message, and commit with **`git: commit`** (`Cmd+Enter` on macOS, `Ctrl+Enter` on Windows/Linux). The terminal equivalent is two commands:

```
git add .
git commit -m "Add price calculation"
```

`git add .` stages every changed file; `git commit -m "..."` saves the snapshot with your message.

When you need it: every time you finish a meaningful piece of work. Commit often, with short honest messages.

Push

Push sends your commits from your computer up to GitHub, so the online copy matches your local one.

In Zed you run **`git: push`** from the command palette.

Because every project in this course is cloned from GitHub, git already knows where "up" is; there is no extra connection step. The terminal equivalent is:

```
git push
```

When you need it: whenever you stop working, so your partners (and a safe online backup) have your latest commits.

Pull

Pull brings commits *down* from GitHub into your computer, so you get whatever your partners pushed.

In Zed you run **`git: pull`** from the command palette. The terminal equivalent is:

```
git pull --no-edit
```

When you and a partner have both committed, a pull has to merge the two histories, and the merge needs a message. `--no-edit` lets git write that message itself. Without it, git opens a text editor inside the terminal and waits (see Troubleshooting).

When you need it: every time *before* you start working. This is the single most important habit when you share a repo. Pull first, and you almost never run into trouble.

Clone

Clone makes a full copy of a repository from GitHub onto your computer, including all its history. This is how every project in this course arrives on your computer: the one you created on GitHub yourself, and a partner's.

In Zed, open the command palette and run `git: clone`, paste the repository's web address (its HTTPS URL, which looks like `https://github.com/name/project.git`, copied from the green *Code* button on GitHub), and pick the folder where you keep course work. Zed then opens the cloned folder as your workspace. The terminal equivalent is:

```
git clone https://github.com/name/project.git
```

When you need it: once per project, when you bring your own new repo down or join a partner's. After cloning you have the repo locally and can commit, push, and pull like normal.

GitHub setup and authentication

Before any push or pull can reach GitHub, your computer has to prove it is allowed to (this is **authentication**). Signing in to the Zed app itself does not do this, it only unlocks Zed's own collaboration and AI features. Git needs its own separate setup, and the reliable way to do it is the GitHub CLI, a small terminal tool called `gh`. You do this once per computer.

> ⚠ **Warning**
> Signing in to Zed is not the same as authenticating git. Even fully signed in to Zed, a push will fail until you complete the `gh` steps below. If a push ever asks for a password or reports a permission or authentication error, the fix is to redo these steps, not to sign in to Zed again.

One-time setup

1. **Install git itself.** Check with `git --version` in a terminal.
   - macOS: if git is missing, that command offers to install it. Accept.
   - Windows: `winget install Git.Git` (or the installer from git-scm.com).

   Then **close and reopen the terminal** (an open one does not see new programs) and re-run `git --version` to confirm you see a version number.
2. **Install `gh`**, the GitHub CLI. Windows: the `.msi` from cli.github.com, or `winget install GitHub.cli`. macOS: the file ending in `macOS_universal.pkg` from the latest release, or `brew install gh` if you use Homebrew. Close and reopen the terminal, then check with `gh --version`. Installing `gh` does **not** install git itself. That was step 1.

3. **Log in.** Run this in a terminal:

   ```
   gh auth login
   ```

   Answer the prompts (arrow keys to choose, Enter to confirm): choose GitHub.com, choose HTTPS as the protocol, and authenticate through the browser (it shows you a short code to type into a GitHub page). When it asks *"Authenticate Git with your GitHub credentials?"*, answer **yes**. This runs `gh auth setup-git` for you, which is the step that lets git push and pull. (If you answered no by mistake, you can run it yourself afterwards:)

   ```
   gh auth setup-git
   ```

4. **Check it worked:**

   ```
   gh auth status
   ```

   It should report that you are logged in to github.com and that git operations use the https protocol.

5. **Tell git how to combine work when you pull:**

   ```
   git config --global pull.rebase false
   ```

   This tells git to simply combine your partners' work with yours when you pull. You only set this once. Without it, git may refuse a pull with a confusing message about "divergent branches".

6. **Tell git who you are.** Every commit carries a name and an e-mail address:

   ```
   git config --global user.name "Your Name"
   git config --global user.email "you@example.com"
   ```

   Neither line prints anything when it works. Use the e-mail address of your GitHub account, so that GitHub links your commits to you. Without these two lines, git either refuses to commit (*"Please tell me who you are"*) or makes up an address from your computer's name.

> **Note**
> Signing in to the Zed app (`client: sign in` in the command palette) is optional and only for Zed's collaboration and AI features. It has nothing to do with git push, so do not treat it as part of git setup.

Starting a project: GitHub first

A project in this course begins on the website and comes down to your computer, never the other way round. Four steps:

1. On github.com, click New repository. Give it a name, set it to Private, tick **Add a README file**, and choose the Python `.gitignore` template (it keeps the `.venv` folder out of git). Click Create repository.
2. Copy the repository's **HTTPS URL** from the green *Code* button (`https://github.com/<you>/<repo>.git`). Use HTTPS, not SSH: the login you set up above only works over HTTPS.
3. In Zed, run **`git: clone`**, paste the URL, and pick the folder that `python-labs` sits in, so the project lands **next to** `python-labs` (terminal: `git clone <url>`). A repo inside `python-labs` gets tangled up with the labs. Zed opens the new folder; everything below happens **inside** it.
4. In Zed's built-in terminal (`Ctrl+`` , it opens inside your workspace) run `uv init --no-package`. It adds `main.py`, `pyproject.toml` and `.python-version` next to the README. Every project has its own packages: if your code imports pandas, run `uv add pandas` here. Commit and push, and the project is live on GitHub.

Working as a group

In this course you work in groups of three, and you share one GitHub repository. There are two clear roles.

The repo owner creates the project and invites the partners:

1. Create the project on GitHub, clone it, run `uv init --no-package` inside, commit and push (the four steps above).
2. On GitHub, open the repository's **Settings → Collaborators** and invite your partners by their GitHub usernames. This gives them permission to push. Invite me as well (my GitHub username is on Moodle), so I can read the private repo for grading.

Each partner joins the existing project by cloning it:

1. Accept the invitation (GitHub sends it by email or shows it on the site).
2. In Zed, run **`git: clone`** with the repository's HTTPS URL (terminal: `git clone <url>`).

From then on **all of you** follow the same two habits every session:

- **Pull before you start.** Run **`git: pull`** so you have your partners' latest work.
- **Push when you stop.** Commit, then **`git: push`**, so your partners get your work.

What a conflict looks like

If you and a partner both change **the same lines** of the same file, git cannot decide which version wins. This is a **conflict**. Git marks the spot inside the file with strange-looking lines (`<<<<<<<`, `=======`, `>>>>>>>`) showing both versions.

> ⚠ Warning
> **Do not try to fix a conflict alone in week one.** Bring it to class or e-mail me, we will resolve it together. The one habit that prevents almost all conflicts: **pull before you start working**, so you build on your partners' latest version instead of an old one.

Troubleshooting

Each entry gives the **symptom** first, then the **fix**.

Symptom: a push is rejected with `! [rejected]`, "fetch first", or "the remote contains work that you do not have locally". Fix: a partner pushed commits that you do not have yet. This is completely normal in a group and is *not* an authentication problem. Pull first (**`git: pull`** or `git pull --no-edit`), then push again. If the pull complains about "divergent branches", run the one-time command from the setup section (`git config --global pull.rebase false`), then pull again. If the pull instead reports a conflict, see What a conflict looks like. That pull is exactly when one can appear. Never force anything: no `--force`, no "force push".

Symptom: after a pull, the terminal fills with a text that starts with `Merge branch` and no longer takes commands. Fix: git has opened its text editor, Vim, and waits for a message about the merge. The suggested message is fine: type `:wq` and press Enter. Next time pull with `git pull --no-edit`, and the editor stays closed.

Symptom: you committed, but your partners still do not see your changes. Fix: a commit only saves the snapshot **on your computer**. It reaches GitHub (and your partners) when you push. Run **`git: push`**.

Symptom: a push asks for a username/password, or reports a permission or authentication error. Fix: your machine's git login is not set up (or expired). Redo the GitHub setup above: run `gh auth login` again and answer **yes** to "Authenticate Git with your GitHub credentials?". Do **not** sign in to the Zed app to fix this, that is a different thing.

Symptom: a git command says `fatal: not a git repository`. Fix: usually you are running git in a folder that is not a repo (or above your project folder). Move into your project folder first. In a terminal, `cd my-project`. In Zed, make sure the folder you opened is the cloned project folder that contains your code, not the folder above it.

Symptom: you committed the wrong thing (bad message, or forgot a file). Fix: never rewrite history that others may already have. Two safe options:

- **Only if you have not pushed that commit yet** (it is still just on your computer, so nobody has seen it), you can fix the last commit in place. Stage the correction and amend:

  ```
  git add .
  git commit --amend -m "Better message"
  ```

- If you already pushed it (or you are unsure), simplest of all: just make a **new commit** that corrects the mistake. A tidy extra commit is completely fine in this course, and it never puts your shared history at risk.

Cheatsheet

Operation	What it does	In Zed	Terminal	When you need it
Create repo	Makes a folder that git tracks	New repository on github.com, then `git: clone`	`git clone <url>`	Start of a new project (GitHub first, then clone)
Stage + commit	Saves a labeled snapshot	`git: stage all`, then `git: commit`	`git add .` then `git commit -m "msg"`	Every time you finish a piece of work
Push	Sends commits up to GitHub	`git: push`	`git push`	When you stop working
Pull	Brings partners' commits down	`git: pull`	`git pull --no-edit`	**Before** you start working
Clone	Copies a repo from GitHub	`git: clone`	`git clone <url>`	Once per project: your own, or a partner's
Status / history	Shows changes and past commits	git panel shows changes	`git status` / `git log --oneline`	Any time, to see where you are

Recap

You can now:

1. Explain what a repository, a commit, push, pull, and clone are.
2. Set up your machine once: GitHub authentication with `gh auth login`, the one-time pull setting, and your name and e-mail for commits.
3. Start a project GitHub-first: create it on the website, clone it, `uv init --no-package` inside, commit, push.
4. Work in a group: pull before you start, commit as you go, push when you stop.
5. Recognize a conflict and know the course rule: bring it to class instead of fixing it alone.

Bibliography